



Operator overloading

1. Need of overloading operators
2. Overloading binary operators
3. Overloading unary operators
4. Handling Pre and Post definitions
5. Overloading cin and cout operators
6. Overloading other operators
7. Operators that can not be overloaded

1. Handling complex mathematical expressions can be quite difficult with functions
2. Use of objects is similar to that of built-in data types

- Steps:
 - Define function with name operator <symbol> as member of class
 - Left hand operand used for calling this function and right hand operand available as parameter
 - Write definition that suits the meaning of that operator
 - Return appropriate data type based on the usage of that operator
- Examples:

- Steps:
 - Define function with name operator <symbol> as member of class
 - Left hand operand used for calling this function but no parameter, as there is not second operand
 - Write definition that suits the meaning of that operator
 - Return appropriate data type based on the usage of that operator
- Examples

- Both versions have no parameter but need to be defined differently
- Dummy int is accepted as parameter to post version of the operator
- Compiler passes 0 as dummy data while calling post definition
- Don't use this dummy data in definition
- Examples:
 - ++
 - --

- cin is extraction operator of class istream
- cout is insertion operator of class ostream
- Overloading must be done using global function which is friend of our class
- Accept parameters by reference
- Return same object by reference for using it further in cascaded manner

6. Overloading other operators

- Function call operator ()
- Subscript operator []

7. Operators that can not be overloaded

- sizeof
- Scope resolution
- typeid
- Ternary operator ? :
- Member access operator .
- *